CHAPTER ONE

INTRODUCTION

1.1 Background:

Software architecture has become an integral part of research and practice in the field of software engineering over the decade. Software architecture is very important because it tries to explain the early design decisions of a system and also shows how to satisfy the quality attributes of a system, thus defining the entire quality of a system (Merson, P., & Smith, D., 2003).

According to Fielding (2000), he defined software architecture *"as a representation (or model) at a high level abstraction of the elements of a software system".* The system may contain many levels of construction as well as representing many phases of operation, but each of this has its own software architecture. The architecture of a system contributes effectively to the quality attributes of a system (Kazman, R., & Mark, K., 1998).

While Bass (1998) provides a more detail/definition of software architecture, he defined it as "*the structure or structures of the system, which comprise software components, the externally visible properties of those components, and the relationships among them".*
The software architecture represents the earliest form of software design decisions. These decisions fully affects quality attribute like reliability, modifiability, security, availability, security, performance e.t.c.

The right software architecture can give a way for the successful development of a system and unfortunately the wrong software architecture mostly results to a poorly developed system that fails to meet up with the required vital non-functional quality attribute requirements. These can arise to many problems, such as, it can lead to high maintenance costs of a system (Lawrence, G. J., & Anthony, J. L., 2001). Therefore, there is a need to evaluate the software architecture of a system to ensure that the required quality attributes are met (Piyush, M., & Albert, T., 2004).

The software architecture design and evaluation have been implemented by many organizations around the world as part of requirement for the standard procedure in the development of software. The software architecture evaluation methods had been used as tools for understanding the organizations business goals during the software development lifecycle. An architecture evaluation technique includes actions, calculating and quantifying of different types of architectural elements, environmental factors and architectural limitations.

It is the duty of a software architect to analyze a given architecture and determine its functional and non-functional requirements, the software architecture evaluation methods had proven to be a much cheaper way of preventing disasters to a system (Jihyun, et al., 2009).

Examples of the software architecture evaluation methods are:

- ATAM (Architecture Tradeoff Analysis Method) (Kazman, R., Klein, M., & Clements, P., 2000).
- QAW (Quality Attribute Workshop) (Nord, R.L., Wood, W.G., & Clements, C.P., 2004).
- SAAM (Software Architecture Analysis Method) (Kazman, R., Abowd, G., & Clements, P., 1996).
- ALMA (Architecture Level Modifiability Analysis) (Ionita, M.T., Hammer, D.K., & Obbink, H., 2002).
- FAAM (Family Architecture Analysis Method) (Ionita, M.T., Hammer, D.K., & Obbink, H., 2002).
- CBAM (Cost Benefit Analysis Method) (Lee, Y., & Choi, H., 2005)

In software system, the success of the qualities attributes such as performance, availability, and security depends more on the overall architecture than on the code-level practices such as language choice, detailed design, algorithms, data structures, testing (Lawrence, G. J., & Anthony, J. L., 2001). Algorithms or data structures is also an important tool in determining software quality, but rather such are considered to be less crucial to a system's success when compared to its overall software structure, its architecture (Kazman, et al., 1998).

The Software Engineering Institute (SEI) had adopted the ATAM which is an improved version of SAAM (Software Architecture Analysis Method) for the evaluation of the software architecture; ATAM is an acronym which stands for Architecture Tradeoff Analysis Method. ATAM can be described as one of the methods that are commonly used for evaluating software architecture with respect to its quality attributes; ever since

the adoption of ATAM, the SEI has validated the importance of putting ATAM into practice (Mario, B., Paul, C., Anthony, L., Linda, N., & William, W., 2003).

ATAM evaluates the quality attributes (e.g. reliability, security, real-time performance e.t.c) and can also be used to identify tradeoffs among conflicting quality goals (some quality goals tends to compete with one another) in the software architecture. ATAM can be used to assess almost all the quality attributes compared to other evaluation methods which have specific quality attribute to evaluate i.e., CBAM considers the business expenses, profit and risks as a quality attribute during evaluation (Mario, B., Paul, C., Anthony, L., Linda, N., & William, W., 2003).

ATAM is an architecture evaluation method detects the risks, tradeoffs, and sensitivity points of architectures of a system. ATAM provides an alternative for the improvement of software structures during system design and analysis as well as offering assistance in the correction of those requirements (Jihyun, et al., 2009).

The evaluation involves a team of architects, evaluators and stakeholders; each stakeholder has to put interest into the diverse system properties and goals that represent the diverse structural views of the system. These diverse properties and goals, and their equivalent architectural views are very important; they have to be completely defined, understood, and carefully analyzed. They all provide the basis for reasoning about the correctness and quality of the architecture (Mario et. al., 2003).

The stakeholders must try and understand the problems that might exist between the conflicting quality attributes, these problems are known as “tradeoff”, “sensitive points” and “risk”. **Sensitivity point** *is “a group of components in the system architecture that are vital for achievement of a particular quality attribute*” (Dennis, S., & Paulo, M., 2003).

Tradeoff *is "a sensitivity point that is vital for the achievement of much quality at*tributes" (Dennis, S., & Paulo, M., 2003).

Risk *is* "*a subset of the sensitivity points that may hinder the system from accomplishing its quality goals"* (Dennis, S., & Paulo, M., 2003).

Internet has become a common tool used for worldwide communication; it is an effective way of communicating and sharing information within the present day information era. Internet has become very important in the field of education as it is important to other areas respectively. Social networking sites, discussion group, news groups, electronic mail and chat group are present day communication tools that are used for educational purposes (Som, N., 2006).

E-learning is fast becoming an educational tool for connecting the world, education institutions i.e., universities, colleges and various learning institutions are now substituting the traditional classroom based learning methods with the present day learning management system (LMS). LMS is the designed software used for the purpose of online education. Examples of such software are moodle, sakai e.t.c (Hűseyin, U., Fezile, Ö., & Zehra, Ö., 2006).

In those educational institutions; such as the colleges, universities, LMS has widely been used as a means to improve the educational services and also to reduce educational cost savings. LMS supports many types of learning activities for both teachers and students within the walls of their classrooms as well as outside the classrooms (Nobuo, F., Toru, N., Noriki, A., Hiroki, K., & Kanako, U., 2010)

Learning Zone is a learning management system (LMS) software use by Universiti Utara Malaysia for online education purpose; UUM's learning zone based on Moodle (Modular Object-Oriented Dynamic Learning Environment).

Learning zone enables lecturers to enhance their students' learning by providing an interactive online environment, distributing reading materials and also encouraging collaboration and interaction of both lecturers and their students within and outside the classroom. UUM's learning zone also allows lecturers to create and modify reading materials for their students. Reading materials can be video, sound file, pdf file, excel spreadsheet, web page, or word documents. UUM's learning zone provides a range of wide functionalities which allows services like content creation and delivery, communication, collaboration and assessment. Learning zone provides services such as assignment module, chat module, forum module, lesson module, turnitin module, web meeting module e.t.c for both students and lecturers usage.

1.2 Problem Statement:

UUM's learning zone had just been operating for one semester. The motivation to this study originated from an issue raised by the director of University Utara Malaysia's Computer Center, that Learning Zone has not been evaluated yet.

UUM's learning zone is learning management system (LMS) like any other web based system that contains non-functional requirements. Examples of such non-functional requirements are performance, security, reliability e.t.c, the non-functional requirement may be of poor quality attributes, meaning that the required non-functional requirement are not satisfactorily met. The unsatisfied non-functional requirement that exist in many present-day learning management systems had cost a lot problems, thereby exposing such systems to many threats like information theft, unavailability of services, low productivity, increase in maintenance cost and many more problems can arises.

Thus, the software architecture directly defines the non-functional requirements of UUM's learning zone system and its evaluation will identify its availability of those requirements and the quality of the system in general.

1.3 Research Questions:

- Does the software architecture of UUM's learning zone system meet the satisfied quality attributes?
- Are there any tradeoffs, potential risk and sensitive point to be identified in the architecture of UUM learning zone system?

1.4 Research Objectives:

The main objective of this study is to evaluate the software architecture of UUM's learning zone system. Specific objectives are:

- To verify that the software architecture had met the satisfied the quality attributes.
- To detect tradeoffs, potential risk and sensitive points in architecture of UUM learning zone system

1.5 Scope of the study:

Learning Zone is a learning management system (LMS) software use by Universiti Utara Malaysia for online education purpose; UUM's learning zone based on Moodle a free Open Source software package. Learning Zone enables lecturers to enhance their students' learning by providing an online environment to distribute materials and encourage collaboration and interaction both within and outside the classroom. This study tries explains how to evaluate the software architecture of Learning Zone system using ATAM.

ATAM will show us how to evaluate the architecture of Learning Zone system with respect to the non-functional requirement of the system, certain non-functional requirement i.e. performance, security, reliability and availability will be evaluated and also tradeoffs and potential risks will be checked among these quality attributes. The evaluation of architecture of Learning Zone involves system stakeholders, architects and evaluators. Therefore the study is restricted to architecture evaluation of the Learning Zone system.

1.6 Significance of the study:

The significance of the studies that will be realized is:

- The study will contribute to the field of knowledge and cover research gaps in the field of software engineering.
- The study will help the Universiti Utara Malaysia's computer center to identify key issues related to quality attributes of the learning zone portal and provide suggestion for improvement.

1.7 Organization of Chapters:

Chapter One presents an overview of the study, its problem statement, objective of the research, scope and the significance of the study.

Chapter Two takes an in dept view of the literature review. The literatures related the research were discuss staring from the learning management system (LMS) to the software architecture and there were all reviewed.

Chapter Three explains the methodology that was employed to achieve the research objective. The non-functional requirement of learning zone, its architecture and general procedure for the methodology was explained.

Chapter Four present the research findings and its analysis.

Chapter Five which is the final chapter will provide a discussion of the findings, its limitation and recommendation for future research.

CHAPTER TWO

LITERATURE REVIEW

2.0 Introduction

The background of this study was introduced in the previous chapter; this chapter will focus on the main topic of discussion and related ideas in previous work that is related our study. The software architecture evaluation of learning management system (LMS) is not an isolated issue; it has been subjected to great number of research before and on this basis relevant literature needs to be reviewed, in order to acquaint the previous research and on-going and future research. In this study emphasis will be given to the theories that have relevance to learning management system (LMS), software architecture, software architecture evaluations techniques and its models.

2.1 Software Architecture

Kruchten (2003) defines Software Architecture as "the set of important decisions about the organization of a software system: selection of the structural elements and their interfaces by which a system is composed, behaviour as specified in collaborations among those elements, composition of these structural and behavioural elements into larger subsystem, architectural style that guides this organization".

Perry and Wolf defined Software architecture in a formula form which was later modified by Boehm, Boehm simply defined "Software architecture = {Elements + Forms + Rationale/Constraints}". Many researchers said to have agreed that the "elements" in the formula were connectors and components (Kruchten, P., 2005).

In the year 2000, the Computer Society had approved the IEEE Standard 1471 as a standard for software architecture; this standard had been documented based on an agreement of the excellence of software architecture practice. In this standard, the software architecture concept was defined. IEEE 1471 defines software architecture *"as the fundamental organization of a system embodied in its components, their relationships to each other and to the environment, and the principles guiding its design and evolution"* (Del Rosso, 2006).

Shaw and Garlan (2006) define software architecture conceptually "as involving the description of the elements that compose the system, their interactions, the patterns and principles that guide their composition and design, and the constraints on those patterns".

Software architecture is the study of the general structures in the software systems, the relationship that exist among subsystems and their components (Shaw, 2001).

We can simply say that software architecture is the entire concept of a system, because it comprises of all the component of that system which include externally visible properties of those component as well as the relationship that exit among them. The components of the system are sometimes known as "connectors", the connectors of a system include: subsystems, objects, processes and compilation unit while the structures are known as "views". Views can be define as a collection of models that represent one aspect of a software system, view is applicable to only one system and not to the generalizations across many software systems (Maier, M.W., Emery, D & Hilliard, R., 2001).

Views are examples of viewpoints, viewpoint are used to describe the current state of the architecture (architectural descriptions) in question from the stakeholders perspective and their concerns over the architecture (Shaw, 2001).

Example of views according to viewpoint IEEE 1471 ontology include: Functional (logic) view, module (or code) view, user action (or feedback) view, development (structural) view, data view. Most of these views are defined in the software architecture reference model e.g., Kruchten's 4 + 1 Model View (Shaw, 2001).

Software architecture mainly deals with the design and implementation of the high-level structures of the software system. Software architecture captures and conserves architects' intentions about a system structure and performance, thereby providing a protection against design decay as a system ages (Kazman, R., & Mark, K., 1998). .

Software architecture is a field that seeks to make use of theoretical design representations; it encourages communication between stakeholders, architects and evaluators. Software architecture provides a medium for reasoning about the design from the earliest system form to its fully implemented system; it establishes a relationship between the goals of a system, been the functional or non-functional and the underlying structure of the system. More formal efforts also exist to ensure that quality attributes are fully addressed in the software architecture (Kazman, R., & Mark, K., 1998).

Software architecture resulted from the assembling of a certain group number of architectural elements in an appropriate form to satisfy the main functionality and performance requirements of the system, as well as some of the non-functional requirements such as performance, extensibility, usability, reliability, modifiability, scalability, portability, security and availability and many other –lities (Kruchten, P., Obbink, H., & Stafford, J., 2006).

The field of software architecture has been categorized into many sub-areas. The International Federation of Information Processing Working Group 2.10 defines these five sub-areas:

- **Architectural design**: How can architecture be produced?
- **Analysis**: Based on the architecture, how can we answer questions about a quality of the final product?
- **Realization**: How can we produce a system based on an architecture description?
- **Economics**: What are the architectural issues that can drive business decisions?
- **Representation**: How can we produce long-lasting artifacts that can communicate architecture to the people? (Kruchten, P., Obbink, H., & Stafford, J., 2006).

Software architecture is a rapidly increasing field of research and practice within the context of software engineering. Software architecture did not suddenly create itself in the early 1990's, but it was at that particular time-frame when the term "software architecture" began to gain popularity and general acceptance from both company and the research community, ever since then the field has attractable been given significant attention to. The foundation of "software architecture" field was laid by a group of programmers like Parnas, Hoare, Perlis, Brooks, Bauer, Dijkstra and others in the late 1960s (Kruchten, P., Obbink, H., & Stafford, J., 2006).

Kruchten, Obbink & Stafford (2006) mentioned that the phrase "software architecture" was first referenced in the year 1969 at a conference on software engineering techniques which was organized by NATO (North Atlantic Treaty Organization).

Software architecture started to grow and has experienced many changes as a result of contributions from both the field of industries and academia. The year 1995 saw the publishing of the first series of software architecture evaluation methods by the Software Engineering Institute (SEI) that is Software Architecture Analysis Method (SAAM) (Kruchten, P., Obbink, H., & Stafford, J., 2006).

The Software Architecture field had gain much popularity among the software engineering families, it has lately grown into maturation as addressed by Mary Shaw and Paul Clements of the Institute for Software Research, International Carnegie Mellon University, after they applied the Redwine-Riddle model of technology maturation to the software architecture field, they concluded that the wide concept of software architecture has run the full course of the model and is a fully matured technology (Kruchten, P., Obbink, H., & Stafford, J., 2006).

The field of software architecture was created out of necessity because of the size and complications of the software systems was increasing day by day, software systems can contain hundred thousand or millions lines of code and this was becoming common and ordinary. Garlan (1992) citied that "as the size and complexity of software systems increases, the design problem goes beyond the algorithms and data structures of the computation: designing and specifying the overall system structure emerges as a new kind of problem. This is the software architecture level of design". The design problem of such software system had gone beyond the limits of algorithms and data structures of the computation; the designing and the shape the overall system structure emerges as a new kind of problem, the main concept of software architecture is to reduce complexity in systems structure through abstraction; by giving an abstract description of the system structures (Malan, R. & Bredemeyer, D., 2005).

2.2 Software architecture evaluation and methods:

Software architecture refers to the general structures in the software systems; the relationship that exists among subsystems and their components, the architecture gives an important description of the system. Kazman (2000) cited that "architectures allow or preclude nearly all of the system's quality attributes". The above definition when considers gives us a much clear understanding of the relationship that exist between software architecture and the quality attributes of a system. Kazman, Klein & Clements (2002) *"If architectural decisions determine a system's quality attributes, then it is possible to evaluate architectural decisions with respect to their impact on those attributes.*

Software architecture evaluation had become a common practice within the context of software architecture families of the software systems. Many organizations around the world have implored the evaluation of software architecture as part of the requirement for standard procedure in the development of software. Software architecture evaluation helps to bring out the organizational business goals as well as addressing the quality of the system (Kazman, R., Klein, M. & Clements, P., 2002b).

Software architecture evaluation involves a series of action like, the calculation and quantifying of different types of architectural elements, environmental factors and architectural limitations. Software architecture evaluation is a series of human-centered activities, it involves a team of evaluators which include architects, developers, system analysts as well as stakeholder (management, customers), the stakeholder work hand-in-hand with the evaluation team as their job is to clearly define their business needs of the organization while the software architects' duty is to collectively put together architectural decisions that will be marked as the architecture of the software system and

also to analyze a given architecture and find out its functional and non-functional requirement, the architecture decisions are collected based on stakeholders description (Kazman, R., Klein, M. & Clements, P., 2002b)..

Software architecture evaluation is called “cheap insurance” because it is a much cheaper way of preventing disasters to a software system. The evaluation reduces cost, time and risk-mitigation to a software system during software development lifecycle. Therefore, software architecture evaluation is a very important factor in determine the success or failure system (Shaw, 2001).

Kazman (2000) mention that using any of the quality attributes based-architecture evaluation methods, one can easily predict the list of all the properties and characteristics that a system possesses.

There are numerous number of new software architecture evaluation techniques that have been developed by various groups in the field of academia, these methods have been reviewed by the international working group on Software Architecture Review and Assessment (SARA), the SARA were able to review and publish all the existing evaluation techniques in the form books for research purposes (Ionita, M.T., Hammer, D.K., & Obbink, H., 2002).

The software architecture evaluation methods adopted by the Software Engineering Institute (SEI) include:

- ATAM (Architecture Tradeoff Analysis Method) (Kazman, R., Klein, M., & Clements, P., 2000).
- QAW (Quality Attribute Workshop) (Nord, R.L., Wood, W.G., & Clements, C.P., 2004).
- SAAM (Software Architecture Analysis Method) (Kazman, R., Abowd, G., & Clements, P., 1996).
- ALMA (Architecture Level Modifiability Analysis) (Ionita, M.T., Hammer, D.K., & Obbink, H., 2002).
- FAAM (Family Architecture Analysis Method) (Ionita, M.T., Hammer, D.K., & Obbink, H., 2002).
- CBAM (Cost Benefit Analysis Method) (Lee, Y., & Choi, H., 2005)

The overview the above methods and the comparison between the methods are given below:

2.2.1 ATAM (Architecture Tradeoff Analysis Method):

ATAM is a scenario-based software architecture evaluation method used for measuring most of all the quality attributes of a software system, ATAM can assess quality attribute such as: modifiability, security, portability, reliability, performance and many more. Moreover, ATAM also discover tradeoffs that may occur among disputing quality attributes because of the interdependencies that exist between certain quality attributes and also the potential risk to the software system. (Ionita, M.T., Hammer, D.K., & Obbink, H., 2002).

The Architecture Level Assessment Method (ALMA) can also be used to carry out measurement of risk performed by ATAM. However, the tradeoff analysis between the different qualities attributes is most excellently expressed in ATAM; the ATAM is known to be an improved version of SAAM. The ATAM divided into four main phases namely: scenario and requirements gathering, architectural views and scenario realization, model building and analysis, and tradeoffs (Ionita, M.T., Hammer, D.K., & Obbink, H., 2002).

2.2.2 SAAM (Software Architecture Analysis Method):

SAAM was the first scenario-based software architecture evaluation method to be introduced by SEI. It was primarily created to assess the modifiability of a given architecture. SAAM had demonstrated to be a very useful tool for assessing many quality attributes, quality attributes such as modifiability, performance, extensibility, portability and reliability. SAAM tries to identify the strong or weakness points of a given architecture as well as the points that determine if the given architecture has fail to meet up with the modifiability requirements (Ionita, M.T., Hammer, D.K., & Obbink, H., 2002).

SAAM can be used to grade the modifiability of two or more different candidate architecture provided they have the same functionality when compared. ALMA which is also a scenario-based software architecture evaluation method can be used appropriately for modifiability assessment of a given software architecture. SAAM involves a series of steps namely: Develop scenarios, describe architecture, classify and prioritize scenarios, individually evaluate indirect scenarios, assess scenario interaction and create an overall evaluation (Ionita, M.T., Hammer, D.K., & Obbink, H., 2002).

2.2.3 ALMA (Architecture Level Modifiability Analysis):

ALMA is a scenario-based software architecture evaluation method used for measuring of modifiability quality attribute of a given software architecture. ALMA was initially developed to test the architecture for Business Information Systems (BIS) and Embedded Systems (ES). ALMA can be used for both predicting maintenance cost and risk evaluation of a given architecture (Ionita, M.T., Hammer, D.K., & Obbink, H., 2002).

ATAM can also be used as an alternative for ALMA with respect to modifiability assessment, since both methods are very similar; they are both use scenarios for the assessment of the quality attributes. ALMA has five step involved in its process namely: Set the analysis goal, describe the software architecture, elicit change-scenarios, evaluate the change-scenarios and interpret results (Ionita, M.T., Hammer, D.K., & Obbink, H., 2002).

2.2.4 FAAM (Family Architecture Analysis Method):

FAAM is a software architecture evaluation method used for architecture evaluation of the Management information system (MIS) families; FAAM focuses mainly on this two related quality attribute namely: interoperability and extensibility (Ionita, M.T., Hammer, D.K., & Obbink, H., 2002).

ATAM can also be used as an alternative for FAAM, only difference between FAAM and ATAM is the scope of the evaluation, FAAM is strictly used for the evaluation of Management information systems families compared to ATAM which is used for a whole range of software architecture. FAAM has been intended purposely for the evaluation of interoperability and extensibility of the Management information systems families. FAAM involves six steps namely: Define the assessment goal, prepare system-quality requirements, prepare architecture, review / refine artifacts, assess architecture conformance, report results and proposals (Ionita, M.T., Hammer, D.K., & Obbink, H., 2002).

2.2.5 CBAM (Cost Benefit Analysis Method):

CBAM is entirely different from the rest of the architecture evaluation methods, CBAM has bridge two domains together that is architecting and the business of the organization. CBAM is used for evaluation of the architecture with addition of costs expenses (Money), unexpected budgets which are considered as quality attributes and also tradeoffs during the planning of the software system development lifecycle (Ionita, M.T., Hammer, D.K., & Obbink, H., 2002).

CBAM considers the expenses, profit and risks to be very important quality attribute in the way other evaluation methods considers modifiability, performance, extensibility, portability and reliability as quality attributes.

The expenses, profit and risks are very much important quality attributes to be considered when the architectural decisions are being made. CBAM consist of five steps namely: Set the analysis goal, describe the software architecture, elicit change-scenarios, evaluate the change-scenarios, and interpret results (Ionita, M.T., Hammer, D.K., & Obbink, H., 2002).

Method	Assessed Quality	Metrics and Tools Support	Process Description	Strengths	Weaknesses	Systems Type Applicable for
SAAM	Modifiability	Scenario classification (direct vs. indirect ones)	Reasonable	Identifying the areas of high potential complexity Open for any architectural description	Not a clear quality metric Not supported by techniques for performing the steps	All
ATAM	Modifiability	Sensitivity Points, Tradeoff Points Supported by ATA Tool [10]	Good	Scenarios generation based on Requirements Applicable for static and dynamic properties Quality utility tree	Requires detailed technical knowledge	All
CBAM	Costs, Benefits, and Schedule Implications	Time and Costs	Reasonable	Provide business measures for particular system changes Make explicit the uncertainty associated with the estimates	Identifying and trading costs and benefits can be done by the participants in an open manner	All
ALMA	Modifiability	Impact estimation, Modifiability prediction Model,	Reasonable	Scenario generation stopping criterion	Restricted set of case studies Concentrates on static properties	Business Information Systems
FAAM	Interoperabilit y and	Various specialized tables and Diagrams	Very good	Emphasis on empowering the teams in applying the FAAM session	Only partially proven in one particular environment	System Families

Figure: 2.1 a summary of software architecture evaluation methods adopted from (Ionita, M.T., Hammer, D.K., & Obbink, H., 2002).

2.3 Learning Management System (LMS):

Learning management systems (LMS) is a broad term used to define the collections of software tools that are used for aiding, facilitating and managing of educational materials for the purpose of learning activities and teaching services (Som, 2006).

Hall (2001) defined Learning Management System (LMS) "*as software that automates the administration of training events*". Lonn and Teasley (2009) simply defined as LMS as online system that allows users to share vast number of information as well as collaborating together via online (internet).

All Learning Management Systems should be able register users, manage the log-in of the registered users, track the courses in a form of catalogs, record data from learners, and provide reports to management for decision making. Vrasidas (2004) described that the main tasks that are associated with LMS includes administration of courses and managing the courses, and pedagogy as well as teaching and learning.

In the normal educational settings, learning management systems has facilitated the improvement of the speed and efficiency of the educational processes; it has established communication among fellow learners, as well as lecturers and students. The use of LMS in the distance learning programs had pave way for education institutions to fully maximize their value, maximization of value occurs by enabling flexible access to its resources and services. Some examples of LMSs are: WebCT, FirstClass, Moodle, Lotus Learning Space, Blackboard, and Desire2Learn (Som, 2006).

Vollmer (2003) mentioned that learning management systems had almost reached its availability in the late 1990s with mostly Blackboard and WebCT being adopted by universities and other educational institution for e-learning purposes.

Blackboard (2006) discussed that some years back specialized service providers like Questionmark, CourseGenie, and articulate carried out enhanced testing, evaluation and content development tools, by replacing most of the tools that are included in many LMS (George, 2006). This action forced many LMS vendors to make some changes thereby providing their partners with priority status in both developing and integrating of the third-party tools. Among the numerous types of e-learning tools, LMS is the most popular e-delivery means that is adopted and widely used by institutions of learning.

Smith and Rupp (2004) stated that due to highly growing number of students, usage of online education has fast become increasingly common in most of the educational institution.

Adams et al (2005), mentioned that California State University initial conducted review on some of the LMS, i.e., Sakai, WebCT, Desire2Learn, Blackboard and also including open source systems i.e., lotus. After the reviewing, all LMS were been classified as ineligible, except for the two which was WebCT and Blackboard. Later WebCT was unanimously selected. Systems were disqualified due to numerous number of reasons, such reasons includes: previous scale of integration, incompatible with "campus data center standards" limited feature sets, limited ease of use, open source movement still in infancy, and lack of confidence in product support by an LMS vendor.

According to Wyles (2004), the evaluation of Learning Management System was very effective in the matching tools like email, bulletin boards, chats, quizzes, tutorials, wikis, etc., together.

This was done with the work of Chickering, Ehrmann, and Gamson, they critically cited this concept in a report which was summarized as: "*Educational institutions need more flexibility and control over their e-learning environments to enable different schools, programmes, course, or instructors to select and deploy the most appropriate e-learning tools suited to the pedagogy*". Any LMS selection process should involve a similar match of functionality with the organization's definition of educating and learning (George, 2006).

2.3.1 Software architecture of a learning management system:

The architecture of a learning management system is defined under the standard of IEEE, IEEE 1484.12.1–2002 is the standard for Learning Object Metadata (LOM), it defines the creation of LOM in the World Wide Web Consortium (W3C) (Liu, X., El Saddik, A., and Georganas, N.D. 2003).

The IEEE 1484.12.1–2002 standard have categorized some standard data structures and communication protocols to be among the standard of LMS, these standard data structures and communication protocols must be present to ensure cross-system flows (Interoperability). The standard data structures are categorized into five, namely: Meta data, content packaging, leaner profile, learner registration and content communication (Liu, X., El Saddik, A., and Georganas, N.D. 2003).

2.3.1.1 Metadata: The Learning materials and catalogs must be categorized in a way that it will allow the indexing, storage, searching, and retrieval of learning materials by various tools across various repositories. The data that undergoes such kind of process is known as learning object metadata (LOM).

2.3.1.2 Content Packaging: Content packaging is a standard which allows the transfer of courses from a learning system to another learning system. The standardization of content Packaging is based on the Simple Sequencing specification, and ADL Sharable Content Object Reference Model

2.3.1.3 Learner Profile: Learner profile information should include the learners personal data, assessments of educational performance and the nature of his participation in the current learning system e.t.c . The standardization of learner profile information is based on the Learner Information Package (LIP) specification.

2.3.1.4 Learner Registration: Learner registration information allows administration to know what course should be made available to the learner, and provides the information about learning participation to the learning environment.

2.3.1.5 Content Communication: During the launching of content, the need for communication between the learner data and series of previous activity information of the content arises.

Liu, El Saddik & Georganas (2003) mention that the standard data structures are integrated into an e-learning system and are presented as the functional model which defines the architecture of a learning management system. The functional model is represented below:

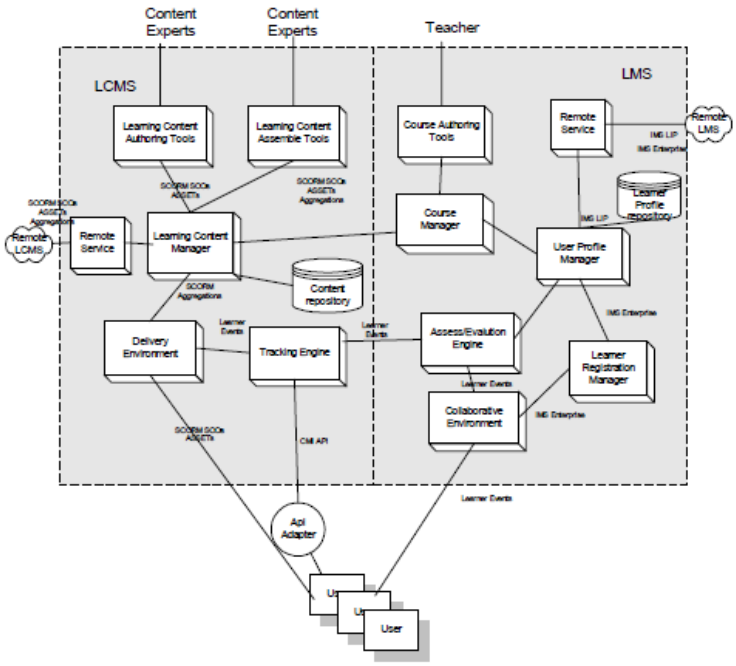

Figure 2:2: architecture of a LMS adopted from (Liu, X., El Saddik, A., and Georganas, N.D. 2003)

2.4 Moodle:

Moodle is an acronym which stands for Modular Object-Oriented Dynamic learning environment. Moodle was started in 1999 by Martin Dougaimas (a Phd candidate in Education who has an educational background in computer science), moodle version 1.0 was the first version to be released in the year 2002 in August (Klaus, 2005). Moodle is an open source software (OSS) meaning that the software free to download and install at no cost under the agreed terms of the general public license (GNU), moodle is supported by group programmers and also the user community (Ajlan, A., & Hussein, Z., 2007).

According to the Moodle homepage (http://moodle.org), the numbers of registered users continue to grow day-by-day, Moodle is using 75 different languages in over 160 countries. Recently there are about 38,000 web sites with over 16,000,000 registered users. Moodle's popularity is growing for many various reasons and one of such is its ability to run on many platforms (Windows, Solaris, Linux, Max OS and UNIX).

Moodle contains modules (designs that allows users to add various types of learning activities) that are used for both for synchronous and asynchronous mode of e-learning, example of such are: m--learning, events, forums, blogs, chats, e-mails, wikis and so on (Britain, S., & Liber, O., 2004).

2.4.1 UUM learning zone:

Learning zone is a learning management system (LMS) software use by Universiti Utara Malaysia for online education purpose; UUM's learning zone based on Moodle (Modular Object-Oriented Dynamic Learning Environment) a free Open Source software package. UUMs Learning Zone is a version 1.9 Moodle. Learning Zone enables lecturers to enhance their students' learning by providing an online environment to distribute materials and encourage collaboration and interaction both within and outside the classroom. UUM's Learning Zone also allows lecturers to create and modify reading materials for their students. Reading materials can be video, sound file, pdf file, excel spreadsheet, Web page, or word documents. UUM's Learning Zone provides a range of wide functionality which allows content creation and delivery, communication, collaboration and assessment. Learning zone provides services such as assignment module, chat module, forum module, lesson module, turnitin module, web meeting module e.t.c for both students and lecturers usage (UUM LearningZone, 2009).

2.5 Implementation of LMS in UUM:

The implementation of learning management system in UUM started as far back as the year 2000, when the idea of integrating both academic and administrative tools of higher learning into a developed e-learning system came up. The plan to embark on the implementation of e-learning portal started in the year 2001. The University Teaching and Learning Centre (UTLC) was tasked to provide the LMS which will give assistance to the university academic staff in designing, preparing and delivering quality instructional materials for courses offered by Universiti Utara Malaysia that can be accessed via the internet (Ahmad, Ta'a & Abu Bakar, 2006).

The UTLC in conjunction with both Pusat Computer (UUM's computer centre) and E-University Unit formed the University's E-Learning committee which is also a sub-unit of UUM's E-university committee. The major objectives of the E-University committee then were to discover the e-learning policies and be able to obtain a learning management system (LMS) that will be suitable for UUM's requirements needs (Ahmad, Ta'a & Abu Bakar, 2006).

The committee prepared a list of specifications that will feature in the university's LMS after three months (1st January to 31st March 2002) of surveying different types of LMS that are available in the market at that time, example of LMS that were survey include: Blackboard, WebCT, Desire2Learn, Sakai, CA, e.t.c, . Two month later saw a developmental changes (May 2002), that is when the university began to develop a leaning management system known as UUM Learning Care management system with a local company (Ahmad, Ta'a & Abu Bakar, 2006).

However, the Learning Care management system was added as module to the already existing modules in the GroupWeb community portal system, a portal system which provides a range of online services to the university community, services such e-mail, calendar, discussion rooms and many more. The Learning Care management system contains four main component integrated at one interface namely: My Desktop, My Portfolio, Resources and GroupWeb. My Desktop contains the lists of all the portfolios that were created and also the portfolios in which you can subscribe to while My Portfolio is use to lists all the portfolios that was created by lecturers for their courses and Resources contain all the list of the existing portfolios created inside the system; can also be used for searching and subscribing can to the available portfolios (Ahmad, Ta'a & Abu Bakar, 2006).

Ahmad, Ta'a & Abu Bakar (2006) mentioned that it was in June 2004 that learning care saw a major developmental changes when it was integrated with the UUM's undergraduate and graduate students information systems to facilitate the e-learning in the university community. They also mention that since 2002 a total of 1900 academic staff had partcipated in the training organized by UTLC and eversince then about 75% of the academic staff have been using the Learning Care management system in conducting their courses (Ahmad, Ta'a & Abu Bakar, 2006).

The implementation of the Learning Care management system at Universiti Utara Malaysia is a major success because the primary objectives set at the beginning had been achieved that is the delivery of e-learning to the university community. However, Learning Care management system was facing some short comings, examples to mention a few are: the relevant materials used does not conform to the standards of SCORM (Sharable Content Object Reference Model), which is the standard for any web based learning management system and also the learning materials used are also not

pedagogical standard, it does not support heavy files upload i.e Video and audio files and so many more problems and this made the E-University committee to review those case and find an alternative solutions to the problems facing the Learning Care management system (Ahmad, Ta'a & Abu Bakar, 2006).

The 2009/2010 academic session saw the implementation of the learning zone as the university new LMS, learning zone was a major success when compared to learning zone because it is guided with SCORM standard and contain more interactive features compared to its former and also its an open source under GNU license agreement meaning that the learning zone is of no cost to the university.

2.6 Summary of the chapter:

In this chapter, a brief introduction of learning management system, learning zone, software architecture and their relevant studies. The related work on the study that was done by previous researchers was discussed and this will be a stepping stone for our ongoing research, the methodology that was adopted will be introduced in the next chapter.

CHAPTER THREE

METHODOLOGY

3.1 Introduction

Methodology presents the framework and guidelines for the researcher to go about on how to achieve his work. Methodology is always at the center of any research efforts, it particularly answer the question "how" (Punch, 2006). Thus, a methodology brings out the ideas as well as specifies how the research objectives will be achieved.

The methodology that will be used in this study is ATAM (Architecture Tradeoff Analysis Method). The ATAM is a standard spiral model which takes one to a more complete understanding of the system and also reduces risk at the same time.

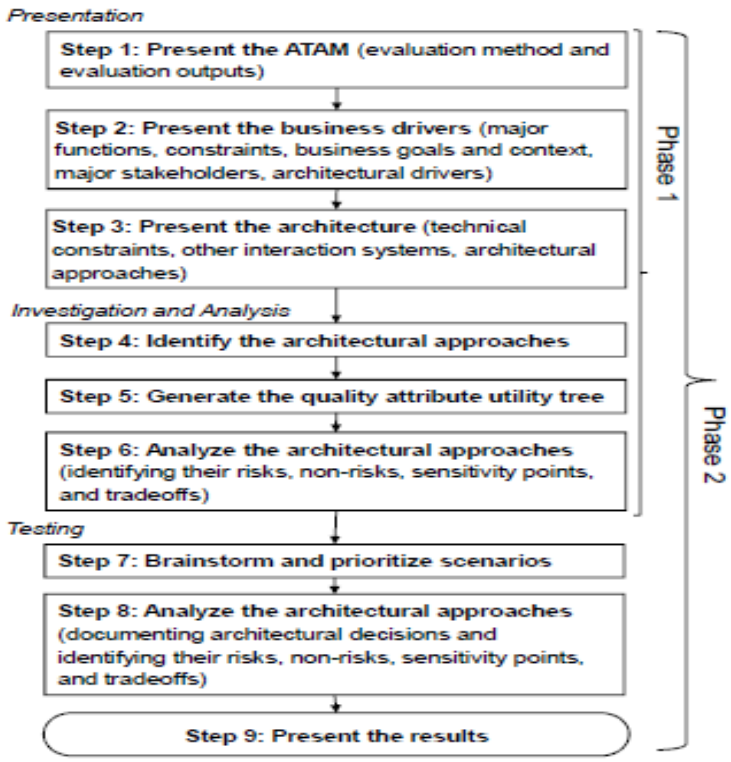

Figure 3.1: ATAM (Jihyun, et.al, 2009).

Fig 3.1 displays our adopted methodology, the main reason behind selecting ATAM as a methodology is that; ATAM can be used to determine and asses the most of the quality attributes of a system. Therefore, evaluating the software architecture of UUM learning zone using ATAM with respect to multiple quality attributes will determine the general quality of the UUM learning zone (Kazman, R., Klein, M., & Clements, P., 2000a).

ATAM is a useful process in the confirmation and evaluation of the quality attributes of a software right from the early stages in the software development life cycle. ATAM is a scenario based software architecture evaluation method which involves series of activities that are conducted by the analyst as well as involvement of stakeholders in the identification of business goals, brainstorming and mapping of the scenarios into the system architecture. All the scenarios are generated based on quality attribute requirements of the system (Kazman, et al, 1998).

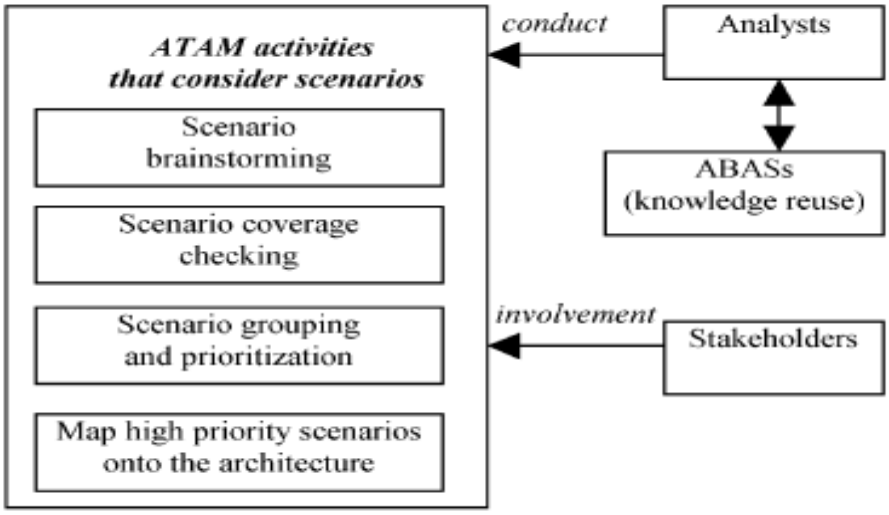

Figure 3.2: ATAM activities that considers it as a scenario-based evaluation (Liliana, D., & Eila, N., 2002)

3.2 Presentation phase:

This the first phase in ATAM, this phase involves primarily the introduction of ATAM as an evaluation method and its expected output at the end of evaluation to stakeholders. The phase also involves the presentation of business drivers (business goals), limitations, main functions and ideas and architectural drivers (Jihyun, et.al, 2009).

Presentation phase consist of three steps namely; present of ATAM, present of business drivers and present of system architecture.

3.2.1 Step One (Present of ATAM)

This involves the presentation of ATAM and its output to the assembled participants by the evaluation leader; the participants involve a collection of evaluators, architects and stakeholders. The evaluation leader tries to explain the expectations to the stakeholder and is expected to answer any questions they may likely ask. In summary, the presentation phase involves:

- A preview of ATAM steps in brief
- The techniques that will be used for analysis: utility generation, architecture elicitation and analysis and scenario brainstorming/mapping.
- The output that will be obtained from the evaluation: the questions that was used to understand/evaluate the software architecture, utility tree, describing and prioritizing the driving architectural requirements, the number of identified architectural approaches and styles, the risks and non-risks that were identified, sensitivity points and tradeoffs that were discovered during the evaluation (Kazman, R., Klein, M., & Clements, P., 2000b).

3.2.2 Step Two (Present of Business Drivers)

In this step, the project manager describes the business drivers of the system; the system that is to be evaluated needs to be explained to all the participants in order to fully understand the process of evaluation. The business drivers include:

- Important functional requirements, quality attribute requirement, business goals and its context of the system.
- The architectural drivers which are the quality attribute that "shape" the system architecture.
- Critical requirements which are the quality attributes that are the most central to the success of the system.
- The preliminary list of each role that will be played by the stakeholders this phase.

3.2.3 Step Three (Present of architecture)

The architect present gives us an insight of the system architecture; the step the architect will present the system architecture at an appropriate level. An appropriate level is a level which contains detail information that tends to asks questions like: how much information needs to be documented and at what much time interval, how much risk does system faces. This is very important because the amount of architectural information available and documentation will have a direct effect on the analysis and also the quality of this analysis (Kazman, R., Klein, M., & Clements, P., 2000b).

The presentation of the architecture should include:

- The architectural style/approaches that were implored to meet the required quality attribute requirements.
- The other systems with which the system must able to interact.
- Technical constraints such as an OS, hardware, or middleware prescribed for use.

From this step, the evaluation team begins to probe the architectural approach of the system to be able to capture the risks (Arnon Rotem-Gal-Oz, 2002). .

3.3 Investigation and analysis

In the investigation and analysis phase, architectural approaches/styles were identified, then quality attribute utility tree is identified and also architectural risk, non-risk and sensitivity points are identified, this phase of ATAM has three steps (Asad, R., Haider, M., Louise, Y., & Ahmed, H., 2009).

3.3.1 Step Four (Identify architectural approach)

In this step, the ATAM tries to focus on the analysis of the system architecture by understanding its architectural approaches/styles. The architectural approaches were identified by the architect, and they are been capture by a group of analysis team, but the architectural approaches are not analyzed by this team. The examples of architectural approaches identified in this step include: client-server architecture, 3-tier architecture, redundant hardware and proxy server.

Buschmann & Shaw (1996) mentioned that emphasis had been laid on the architectural approaches/styles because they characterize the architecture's way of addressing the highly prioritized quality attributes of the system.

These architectural approaches define the vital structures of the system and express the ways in which the system can develop, react to any changes that may occur, survive any external attacks and should be able integrate with other systems (Kazman, R., Klein, M., & Clements, P., 2000b).

3.3.2 Step Five (Generate quality attribute utility tree)

In this step, the entire ATAM team is involved. The evaluation team works hand-in-hand with the architecture team, stake holders (project manager, customer representatives, and users) to identify the highly prioritize quality attributes. This step is very vital because it tries to bring out the most refined important quality attributes goals of the system and it also serves as a guide on how the reminder of the analysis is to be conducted during the evaluation process. The analysis is been done by building a utility tree.

The utility tree generation process leads to the realization of scenarios, scenarios are branches and leaves of a utility tree. Utility tree generation is a process which involves the prioritization of precise quality attribute requirements. The quality attributes requirement derived from utility tree generation gives us a road map to the reminder of the ATAM. The highly prioritize quality attributes requirement generate from utility tree generation includes: performance, availability, security, modifiability e.t.c.

The utility tree gives a guide to the ATAM evaluation team where to focus i.e., where to spend their time, and in which particular place to check for the architectural approaches and their resulting risks, sensitivity points, and tradeoffs that exits.

The scenarios realized from the utility generation are mostly used to understand the highly prioritized quality attribute requirement; they represent the entire stakeholder's interest. The scenarios realized are use case scenarios, growth scenarios and exploratory scenarios (Arnon Rotem-Gal-Oz, 2002).

Examples of scenarios include:

- Use case scenario; occurs when a remote users request a database report from the system via the web via web and receives the report in 5 seconds during the peak period.
- Growth scenario; is when a new data server is added in order to reduce the latency in the scenario from 1 – 3 seconds within 1 person to week.
- Exploratory scenarios; is when half of the servers go down during normal operation without affecting the overall system availability.

All scenarios should be easily specified as soon as they are realized from the utility tree generation (Kazman, R., Klein, M., & Clements, P., 2000b).

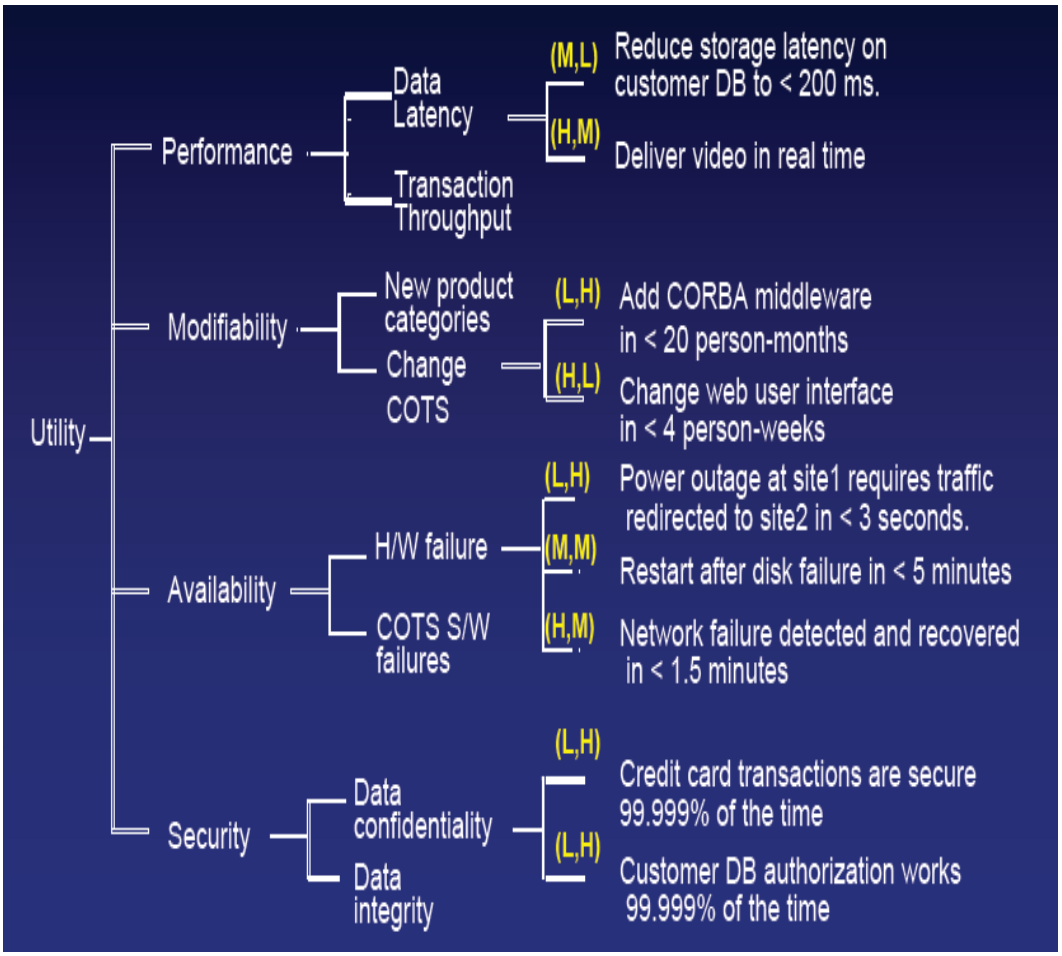

Figure 3.3 Utility tree (Carnegie Mellon University, 2002).

3.3.3 Step Six (Analyze Architectural Approaches)

In this step, the evaluation team then checks the architectural approaches to get the important quality attributes and identify the risks. The main outputs of this phase are the architectural approaches/styles, the frequent asked questions associated them and architects responses to those questions. From these approaches a list of sensitive points, risks and tradeoffs are generated and recorded. The utility tree tries to tell us exactly where to explore in the architecture and also the architect tries to responds to the architectural approach with the much need answers. The questions asked from the helps architectural approaches/styles to:

- Tries to understand the approach
- Always look for well-known weaknesses with the approach
- look for the approach's sensitivity points
- Find and locate the interactions and tradeoffs with other approaches

We can also obtain information from an architect in response to a utility tree scenario that required high availability from a system, example of such is shown below:

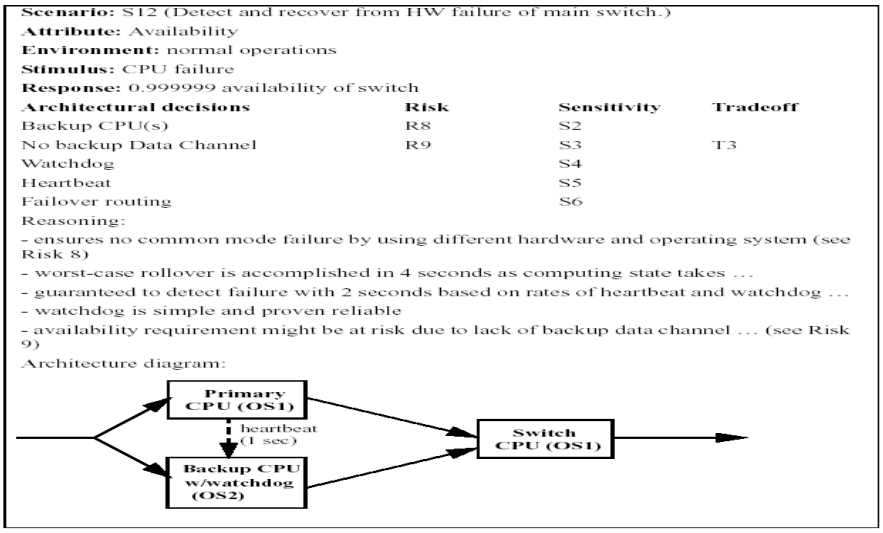

Figure 3.4 Example of Architectural Approach Description (Carnegie Mellon University, 2002).

From the results obtained from this step we can simply identify and record all the risks, sensitivity points, tradeoffs and non-risks of the system architecture.

Sensitivity point is a group of components in the system architecture that are vital for achievement of a particular quality attribute.

Tradeoff is a sensitivity point that is vital for the achievement of many quality attributes.

Risk is a subset of the sensitivity points that may hinder the system from accomplishing its quality goals.

Non-Risk is component in the system architecture which poses no threat to the quality goals.

The sensitivity points, tradeoffs and risks/non-risks are collected together and separated into three different categories. At end of the ATAM, all sensitivity points and tradeoffs points are either categorized to be a risk or a non-risk.

3.4 Testing phase

In the testing phase, stakeholders are involved in brainstorming and evaluation of prioritizing scenarios again. Doing this may discover extra architectural risk, non-risk and sensitivity points.

3.4.1 Step Seven (Brainstorm and Prioritize Scenarios)

The testing phase begins in this step. Scenarios are the main force that generates brainstorming and discussion, because a lot of stakeholders gather together in the ATAM to participate and discuss.

Here the stakeholders partake in two main scenarios, namely: use case scenarios and change scenarios, the use case scenarios are used to execute some functions by the stakeholders and the stakeholder serves as the end users while the change scenarios are scenarios that can explain any changes to the system.

When all the scenarios are collected, they are subject to be prioritized and the votes are casted by the stakeholders. After the voting exercise the scenarios from the utility tree are ranked as highly ranked scenarios are selected and compared with one another and they are either agreed or disagreed.

The utility tree and the brainstorming clearly define the quality goals of the system. This helps the ATAM to fully understand the needs of the stakeholders (Kazman, R., Klein, M., & Clements, P., 2000b).

3.4.2 Step Eight (Analyze Architectural Approaches)

After the collection and analysis of the scenarios, the highly ranked prioritized quality attributes are been mapped to the match the architectural description. This phase is a testing activity phase; it ensures that scenarios that are generated from the utility tree identify the architectural approach.

This step gives a recap of all the activities involve in step six but it uses the highly prioritized scenarios from step seven any case the step seven did not produce any highly-prioritized scenarios that were not already covered by previous analysis, Step 8 is a testing activity.

3.5 Reporting phase:

The reporting phase is used for the documentation of the result on the sensitivity points, utility trees e.t.c collected during evaluation to the stakeholders; in the reporting phase we have the last step which is the presentation of the result of ATAM.

3.5.1 Step Nine (Present Results)

Finally the ATAM team presents a summary of the findings that were collected during the evaluation to the stakeholders which is in a form of report but sometimes can be a in a verbal report. During the presentation, a recap of all the steps of the ATAM and all the vast information collected in the steps of all the processes involved including: the business goals, quality requirements, constraints, and the overall architecture is summarized and pass over to the stakeholders. The finally a set of output of the ATAM are also present, they include:

- The architectural approaches/styles
- The utility tree
- The scenarios and their prioritization
- The risks and non-risks discovered
- The sensitivity points and tradeoffs points.

3.6 Summary of the chapter

ATAM is the adopted methodology that we implore in this research. The chapter makes an introduction of ATAM as methodology for the evaluation of software architecture, ATAM is used to determine the quality attributes goals of the system with respect to the non functional requirements and this gives us the general quality of the system.

ATAM is divided into two phases namely; phase one and phase two. Phase one is further sub-divided into presentation phase, and investigation & analysis phase while phase two is sub-divided into testing and reporting phase. ATAM consist of series of steps namely; Present of ATAM, Present of Business Drivers, Identify architectural approach, Generate quality attribute utility tree, Analyze Architectural Approaches, Brainstorm and Prioritize Scenarios ,Analyze Architectural Approaches and Present Results.

The detail analysis and results of the software architecture evaluation of UUM learning zone will be presented in the next chapter.

CHAPTER FOUR

RESEARCH FINDINGS AND ANALYSIS

4.1 Introduction

In this chapter, the relevant procedures for the evaluation of UUM learning zone and the research findings from our methodology are presented.

4.2 ATAM of learning zone:

The evaluation of learning zone using ATAM starts with the procedure giving from the methodology. ATAM consist of four phases which are categorized into eight steps, phases include presentation, investigation and analysis, testing and reporting.

4.2.1 Presentation Phase:

Presentation phase consist of three steps namely; present of ATAM, present of business drivers and present of system architecture.

This is the general introduction of the project team, evaluators, architects, customers and stakeholders. Introduction is the first action to be taken, but in our case study we are going to use architect, evaluators, stakeholders and system analyst this because the learning zone is fully implemented system.

We introduce the techniques to be used in our evaluation, they include:

- Utility tree generation to analyze each of the quality attributes in the learning zone.
- Scenario brainstorming to either add or argue about scenarios related to our quality attribute of the learning zone.

Presentation phase is just an introduction of our input and expected output at the end of the session (Kazman, R., Klein, M., & Clements, P., 2000b).

We will present the business drivers of the learning zone they include: the business goals and quality attributes of the learning zone.

The business drivers are business goals of the learning zone they include:

- Enhancing quality teaching and learning within the university community.
- Managing the delivery and administration of programs of learning through via internet.

Quality attributes are properties of a system which have nothing to do with functionality of the system are known as non functional requirement; they define the quality of a system. Examples are reliability, safety, performance, usability, availability, security and many more. Below are descriptions of some of the quality attributes that are present in learning zone:

- Availability: readiness for usage
- Reliability: stability of service
- Safety: non-occurrence of problem on the system
- Maintainability: ability to undergo maintenance and changes

- Security: measurement of the system's capability to resist unauthorized access

In our analysis, we are going to focus on some of these quality attributes of learning zone.

The presentation phase will not see any major changes; the phase actually tries to give us an in-depth idea of our business goals and quality attributes that are related to learning zone.

4.2.2 Investigation and analysis phase:

In this phase, we identify the learning zone's main quality attribute goals, this quality attributes are prioritized and polished, the quality attribute is presented in a utility tree, in presenting the utility tree we will identify the architectural risk, non-risk and sensitivity points if there are any.

4.2.2.1 Generate quality attribute utility tree:

This step is vital because it mainly brings the attention of all the people involved in the evaluation process (stakeholders and evaluators) to the different features of the software architecture that are very important to the success of the software system, This is accomplished with the generation of a utility tree. The utility tree generation process leads to the realization of scenarios, scenarios are branches and leaves of a utility tree. The scenario is demonstration of how the stakeholder interacts with the system. These scenarios are what we going to use to evaluate the architecture (Kazman, R., Klein, M., & Clements, P., 2000b).

The research is going to make a scenario generation which is an important step in utility tree creation; scenarios are generated based on our interaction (Stakeholders) with the system. Below is table form of my scenarios generated from learning zone and the quality attributes it represent.

Table 4.1: scenarios and their related quality attribute

Scenarios	Quality Attribute
Entering a wrong user name and password	Security
Clicking an icon to process information with speed.	Performance
encapsulated, secured e-mail	Security
Adding new web pages to my wiki module	Modifiability
Integrating mybox for my file storage	Modifiability
Downloading of lecture slides	Performance
When Network fails during virtual meet and later recovered	Availability
the system should be able to handle user errors	Reliability

From the above table 4.1, we can now generate utility tree, we then identify the "utility" as the root of the tree, while all the quality attributes identified are structured as the branches and leaves of a utility tree the utility tree. Under each of the quality attributes, specific quality attribute modifications are included which will clearly give explanations of our scenarios.

From the utility tree generation the prioritized all the quality attributes, the prioritization is based on two criteria: the value of each scenario to the success of the system and an rough assessment of the level of difficulty posed by the realization of this scenario The prioritization is normally based on rankings: the scenarios are prioritized as High, Medium and Low which are symbolized as H (High), M (Medium), and L (Low).

Based on the interaction with the learning zone a utility tree for the quality attributes realized from the scenarios was drawing.

4.2.2.1.1 Utility tree for learning zone:

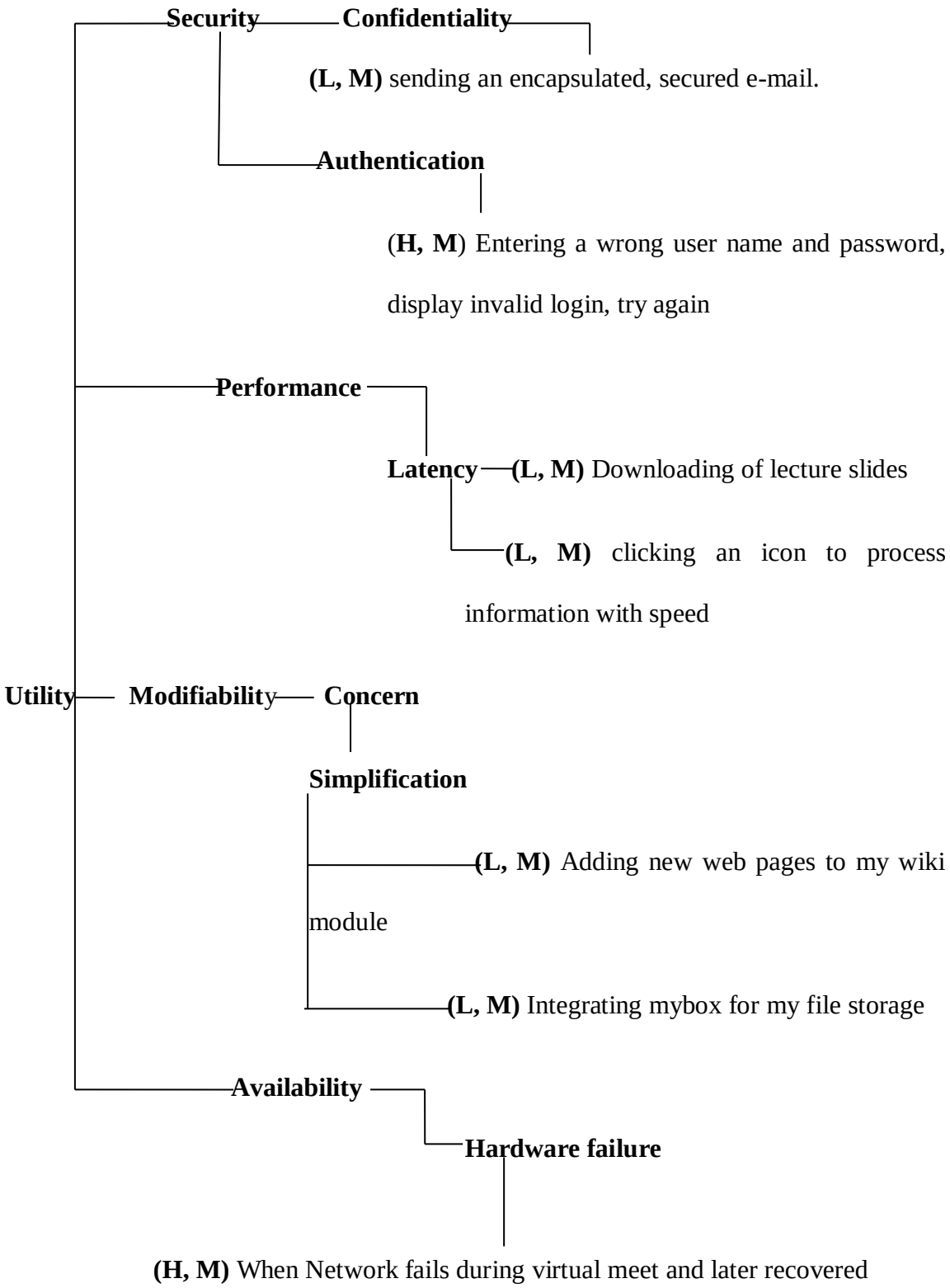

Figure 4.1 utility tree of learning zone

From the above utility tree, the scenarios are prioritized by the (H, M) and (L, M). The (H, M) prioritized scenario signifies that it of high importance to the success of the system, and of medium level of difficulty to the architect while the (L, M) signifies that it of low importance to the success of the system, and of high level of difficulty to the architect. On this basis the scenarios with (H, M) are chosen because of its importance to the system and scenarios with (L, M) are ignored.

4.2.2.2 Analyze Architectural Approaches:

This is the last step of this phase; the output we obtained from our utility tree generation in the preceding step are going to analyze, the analysis is done to find our architectural approaches of learning zone with respect to the related quality attributes. The analyzed software architecture of learning zone based on the quality attributes, and there is a need for an explanation of these quality attribute. From the architectural approaches of learning zone a list of sensitive points, risks and tradeoffs are generated and recorded.

From the utility tree generated, a few lists high priority attributes have been obtained. The highly prioritized quality attribute are: Availability and security while performance and modifiability were not highly prioritized because they are of little significant to the systems goal (Kazman, R., Klein, M., & Clements, P., 2000b). The quality attributes are either prioritized as high or low, for learning zone we have:

Highly prioritized:

(**H, M**) Entering a wrong user name and password, display invalid login, try again **(Security).**

(H, M) When Network fails during virtual meet and later recovered **(Availability)**

Low prioritized:

(L, M) sending an encapsulated, secured e-mail **(Performance)**.

(L, M) Downloading of lecture slides **(Performance)**

(L, M) Adding new web pages to my wiki module **(Modifiabilit**y)

(L, M) Integrating mybox for my file storage **(Modifiabilit**y)

(L, M) clicking an icon to process information with speed (**Performance)**

This step is further sub-divided into four stages:

- Critical investigations of the architectural approaches of learning zone that are pertained to the highly prioritized quality attribute,
- Creation of analysis questions for the highly prioritized quality attribute,
- Giving answers to the analysis questions and
- Identification of the risks, non-risks, sensitivity points, trade-off points in the learning zone (Kazman, R., Klein, M., & Clements, P., 2000b).

4.2.2.2.1 Critical investigations of the architectural approaches:

After the identification of the high priority quality attributes of the learning zone system, the architecture is analyzed to find out how learning zone support these high priority quality attributes. The full investigation of the architectural approaches of learning zone is carried out to see if those high priority qualities attribute requirements are been satisfied.

- **Security:** security is an attribute that stops an action from causing destruction. In the learning zone, when a user tries to log on using his user id and password, it checks for his authentication, if his user name and password are both correct it launches the main page but if one of the two is wrong it displays a message “Invalid login, please try again”. No matter how many

times a user tries to log on wrongly, the main page of learning zone will not be launched. This ensures high-level of security of the architecture.

- **Availability:** The availability is defined as measurement of readiness for usage of a system. In the learning zone, u can use the virtual meet module to organize a share video (Live web cams) chat with your colleague, due to its the scalability nature for users (1-100 persons) u might experience a network failure due congestion and most of the time it takes minimal amount of time to recover. Hence the availability property is been handled correctly.

4.2.2.2.2 Creation of analysis questions:

This involves the creation of analysis questions generated from the high prioritized scenarios from the critical investigations of the architectural approaches. In this project, questions are created from the generated highly prioritized scenarios of the learning zone. The questions asked are related to each of the architectural approach. The questions are:

- How long will the system be available for a virtual meet's web cam live chat when there is a network failure (**Availability**)?
- Can the system authenticate a user if tries to log on with either wrong user id or password (**Security**)

4.2.2.2.3 Answers the analysis questions:

We relatively try to answer the question here, the answers are:

- When you experience a network failure during virtual meet's web cam live chat which is probably due to congestion it take a minimal amount of time for the system to recover and be available again.
- No, no matter how a user tries to log on with invalid user name or password the system will continue to display "Invalid login, please try again".

4.2.2.2.4 Identification of the risks, non-risks, sensitivity points, trade-off points:

Sensitivity point is "a group of components in the system architecture that are vital for achievement of a particular quality attribute". **Tradeoff** is "a sensitivity point that is vital for the achievement of more quality attributes". **Risk** is a "subset of the sensitivity points that may hinder the system from accomplishing its quality goals". **Non-risk** is "a subset of the sensitivity points that allows the system to accomplishing its quality goals" (Dennis, S., & Paulo, M., 2003).

Therefore, the non-risks quality attributes identified from learning zone are: **Security** and **availability.**

The sensitivity points identified from the highly prioritized quality attribute of learning zone are:

- The access of the main page of learning zone is sensitive to a valid user id and password.
- The availability of virtual meet's web cam chat live is sensitive to network failure cause by congestion.

Based on the research, there are no tradeoffs discovered from the learning zone system because there is no any sensitive point for more than one attribute.

4.3 Testing phase:

We are involved in brainstorming and evaluation of prioritizing scenarios again. Doing this may discover extra architectural risk, non-risk and sensitivity points.

4.3.1 Brainstorm and Prioritize Scenarios:

The previous steps represent the interests of the stakeholders and were used to understand the quality attribute requirements. The quality attributes generated from the utility tree are been perceived as an outcome that represent the architect's point of view about the quality attributes of the system. The main goal in this step is to carry every one along that is to say that the entire household of the stakeholder community is involved. The prioritized list of brainstormed scenarios is compared with the prioritized scenarios obtained from the utility generation tree in step five. Here the stakeholders are expected to brainstorm three kinds of scenarios, namely:

- Use case scenarios- are scenarios in which the stakeholder is the user
- Growth scenarios- represents the ways in which architecture's growth is perceived
- Exploratory scenarios- represent extreme forms of growth in the architecture (Kazman, R., Klein, M., & Clements, P., 2000b)
- Examples of such in learning zone include:
- Use case scenarios- Entering a wrong user name and password, display invalid login, try again.
- Growth scenarios- encapsulated and secured e-mail.
- Exploratory scenarios- When Network fails during virtual meet and later recovered in a minimal amount of time.

The activities involved in this step include:

- The collection of scenarios after a stakeholders' brainstorming session
- we ensure that all the scenarios are been prioritized; that is to say that all the scenarios pertaining to the same quality attribute are merged together and a voting session is done to enable the stakeholders to vote for the scenarios they think are most important to the system architecture.
- Each of the given a number of votes which is defined in a form of a formula as shown; Number of votes = 30% X total number of scenarios (Kazman, R., Klein, M., & Clements, P., 2000b).
- After the voting, each of the votes is marked and the scenarios are well-organized according to the total number of vote. Scenarios that were above a cut-off line are selected while the rest are discarded.

The list of prioritized scenarios generated from brainstorming is relatively compared with the prioritized scenarios obtained from the utility tree generation of the learning zone. Each of the brainstormed scenarios are selected and placed under its right branch in the utility tree. In case there is any duplication of scenario in the tree, then the scenario is subjected to a new leaf.

In our project, we only added one prioritized brainstormed scenario that is; the system should be able to handle user errors (**reliability**). We present a list of scenarios in a table form as seen below:

Table 4.2 showing Scenarios, quality attribute and their scenario no.

Scenario	Quality attribute	Scenario No.
Entering a wrong user name and password	Security	1
Clicking an icon to process information wi speed.	Performance	2
Sending an encapsulated, secured e-mail	Security	3
Adding new web pages to my wiki module	Modifiability	4
Integrating mybox for my file storage	Modifiability	5
Downloading of lecture slides	Performance	6
When Network fails during virtual meet	Availability	7
the system should be able to handle user errors	Reliability	8

Therefore, the Number of votes = 30% X total number of scenarios will be:

Number of votes = 0.3 x 8 = 2.4 (approx. 3) because we have to round up to a whole number. Therefore each of the stakeholders has 3 votes to cast on any scenarios of their choice. In this project, stakeholders use intuition to cast the votes on the scenarios.

Table 4.3: showing number of votes casted by the stakeholders on each quality attribute

Scenario no.	Scenario	Quality attributes	Stakeholder Votes
1	Entering a wrong user name and password	Security	3
2	Clicking an icon to process information with speed.	Performance	3
3	Sending an encapsulated, secured e-mail.	Security	2
4	Adding new web pages to my wiki module	Modifiability	1
5	Integrating mybox for my file storage	Modifiability	1
6	Downloading of lecture slides	Performance	2
7	When Network fails during virtual meet and later recovered	Availability	3
8	the system should be able to handle user errors	Reliability	2

After the voting session, the quality attribute for the scenarios are been merged and also tabulated.

Table 4.4 showing scenarios merged with their quality attributes.

Scenario no.	Scenario	Quality attributes
1	Entering a wrong user name and password	Security
3	Sending an encapsulated, secured e-mail	
2	Clicking an icon to process information with speed.	Performance
6	Downloading of lecture slides	
5	Integrating mybox for my file storage	Modifiability
4	Adding new web pages to my wiki module	
7	When Network fails during virtual meet and later recovered	Availability
8	the system should be able to handle user errors	Reliability

4.3.2 Analyze Architectural Approaches:

In this step, there is a need to analyze all the highly voted quality attributes identified from the Brainstorm and Prioritize Scenarios step. In analyzing the highly voted quality attributes, the stakeholders ensure to find the architectural approach that is related to these quality attributes and also to ensure if the architectures support these highly voted quality attributes or not.

After all these, a repetition of similar procedure carried out during "Analyze Architectural Approaches", the only difference here is that we using the highly voted brainstormed quality attributes of the learning zone rather than the high prioritized scenarios generated from utility tree (Kazman, R., Klein, M., & Clements, P., 2000b).

From table 4.3, the highly voted attributes are: security, availability and performance. Previously the same procedure for security and availability in analyze architectural approaches step was done and it can also be done with performance attribute.

This step is further sub-divided into four stages:

- Critical investigations of the architectural approaches that are pertained to the highly voted quality attribute to identify the risk,
- Creation of analysis questions for the highly voted quality attribute,
- Giving answers to the analysis questions and
- Identification of the risks, non-risks, sensitivity points, trade-off points.

Critical investigations of the architectural approaches

Performance: in the learning zone, the time taken to perform a task is minimal, i.e clicking an icon to process information with speed. Some task could have more time depending on the components of the system involved, example downloading a high capacity data but still the time is fast and minimal. Therefore the issue of performance is satisfied.

Creation of analysis questions

Does the architecture process any task with the fastest speed? (**Performance**).

Giving answers to the analysis questions

Yes, it can process the task in the fastest speed.

Identification of the risks, non-risks, sensitivity points, trade-off points

- Performance is a non risk
- Sensitivity point: The average speed used to perform a given task is sensitive to the number of components involved during the processing of that task.
- Trade-off points: There is no tradeoff.

4.4 Reporting phase:

4.4.1 Present Results:

Finally the ATAM team presents a summary of the findings that were collected during the evaluation to the stakeholders which is in a form of report but sometimes can be a in a verbal report. The set of output of the ATAM are:

- The architectural approaches/styles
- The utility tree
- The scenarios and their prioritization
- The risks and non-risks discovered
- The sensitivity points and tradeoffs points.

Although this is the report phase, architectural approaches/styles, utility tree, the scenarios and their prioritization all result have been explained in their respective phases, they are no risk and tradeoff identified while the sensitivity points are:

- The access of the main page of learning zone is sensitive to a valid user id and password.
- The availability of virtual meet's web cam chat live is sensitive to network failure cause by congestion.
- The average speed used to perform a given task is sensitive to the number of components involved during the processing of that task.

CHAPTER FIVE

DISCUSSION AND CONCLUSION

5.1 Introduction:

This chapter wills gives a review of our research findings, limitations encountered during this research. And we are going to take a look back at our research objectives and see if we have achieved those objectives.

5.2 Findings:

The present study aims at evaluating the software architecture of learning zone with respect to its quality attributes, these quality attributes are important to the success of a system; software architecture evaluation is a “cheap” way of preventing disaster. The two research objectives will be answered thus:

5.2.1 Research objective 1:

The first research question aims at verifying if the software architecture has met the satisfied the quality attributes. To answer this objective we take a closer look at Table 4.1:

Table 4.1: scenarios and their related quality attribute

Scenarios	Quality Attribute
Entering a wrong user name and password	Security
Clicking an icon to process information with speed.	Performance
encapsulated, secured e-mail	Security
Adding new web pages to my wiki module	Modifiability
Integrating mybox for my file storage	Modifiability
Downloading of lecture slides	Performance
When Network fails during virtual meet and later recovered	Availability
the system should be able to handle user errors	Reliability

From the above (table 4.1), most of the quality attributes that are critical to the success of a system are identified. The entire quality attribute identified are important in achieving the business goals of learning zone.

5.2.2 Research objective 2:

The second objective is to detect tradeoffs, potential risk and sensitive points from the architecture of UUM learning zone system. To answer this objective we take a closer look at the present result step in reporting phase of ATAM in chapter four. In the reporting phase, we mentioned that there are no risk identified from both highly prioritized quality attribute and highly voted quality attribute. There are no tradeoff points identified and for the sensitivity point we have identify are:

- The access of the main page of learning zone is sensitive to a valid user id and password.
- The availability of virtual meet's web cam chat live is sensitive to network failure cause by congestion.
- The average speed used to perform a given task is sensitive to the number of components involved during the processing of that task.

5.3 Limitations and recommendations:

The study has some short comings as it is a normal phenomenon in conducting research. First, the study needs to be a full house workshop; it involves a collective group of stakeholders hand pick from various expertise groups. For ATAM is need for architects, customers, evaluation team to be present and it takes much long time probably days or week to come up with evaluation results.

Secondly, at the time the research was conducted, UUM's learning zone was undergoing system maintenance, access to some of the facilities were difficult to attained and also time is a major factor here as it is much need if we are to conduct the evaluation of all the quality attributes identified from learning zone.

To fully evaluate the software architecture of learning zone using ATAM, generation of more quality attribute is required from the architecture of the system so that all types of the present are fully explored and evaluated. Since ATAM calls for improvement of the business goals of a system, the learning zone should be evaluated from time-to-time to determine its current status.

5.4 Conclusion:

The purpose of this study was to evaluate software architecture of UUM's learning zone with respect to its quality attributes using ATAM. Evaluating the UUM's learning zone architecture will give us an in dept look at the quality attribute that brings the major success of the system and also the quality attributes that poses as a risk to the system. The results have shown us that they were no risk and tradeoff involved in the architecture and a few sensitivity points were identified**.**

Reference

Ahmad, Ta'a & Abu Bakar (2006). Development and Implementation of an LMS: Universiti Utara Malaysia's Experience, Universiti Utara Malaysia. Retrieved on 13th June, 2010 from www.asiapacific-odl.oum.edu.my/C33/F241.pdf

Ajlan, A., & Hussein, Z. (2007). E-Learning (MOODLE) Based on Service Oriented Architecture. Retrieved on 7th June, 2010 from www.tech.dmu.ac.uk/STRL/research/publications/.../2007-23.pdf

Arnon Rotem-Gal-Oz. (2002). ATAM: Architecture Tradeoff Analysis Method. Retrieved on 4th June, 2010 from www.rgoarchitects.com/Files/ATAM.ppt

Asad, R., Haider, M., Louise, Y., & Ahmed, H. (2009). Security Characterization for Evaluation of Software Architecture using ATAM. IEEE 2009

Britain, S., & Liber, O. (2004). A Framework for the Pedagogical Evaluation of eLearning Environments. Retrieved on 5th June, 2010 from www.elearning.ac.uk/resources/VLEFullReport08.doc/download

Carnegie Mellon University (2002). Architecture Tradeoff Analysis Method (ATAM). *CA Software Engineering Institute, Carnegie Mellon University Pittsburgh, PA, U.S.A. 15213*. Retrieved on 8th June, 2010 from www.seal.ifi.uzh.ch/fileadmin/User_Filemount/.../swa-04-ATAM.pdf

Del Rosso (2006). Continuous evolution through software architecture evaluation: a case study. Journal of software maintenance and evolution: research and practice j. Softw. Maint. Evol.: res. Pract. 2006; 18: Page 351–383

Dennis, S., & Paulo, M (2003). Using Architecture Evaluation to prepare a large Web Based System for Evolution. *Proceedings of the fifth IEEE international workshop on website evolution (WSE'03)*. IEEE 2005.

George, S. (2006). Learning or Management System? A Review of Learning Management System Reviews. Retrieved on 5th June, 2010 from www.connectivism.ca/?p=243

Hűseyin, U., Fezile, Ö., & Zehra, Ö (2006). An Evaluation of Open Source Learning Management Systems. According to Learners Tools. Retrieved on 31st May, 2010 from www.formatex.org/.../An%20Evaluation%20of%20Open%20Source.pdf

Ionita, M.T., Hammer, D.K., & Obbink, H. (2002). Scenario-Based Software Architecture Evaluation Methods: An Overview. Retrieved on 10th June, 2010 from www. *citeseerx.ist.psu.edu/viewdoc/download?doi=10.1.1.102*

Jihyun, L., Sungwon, K., Hyungchul, C., Byungnyun, P., & Choonbong, L. (2009). Analysis of Van-Core System Architecture- *a case study of applying the ATAM*. IEEE 2009.

Kazman, R., Abowd, G., & Clements, P. (1996). Scenario-Based Analysis of Software Architecture. Retrieved on 16th June, 2010 from www.cs.uwaterloo.ca/research/tr/1995/45/CS95-45.pdf

Kazman, R., & Mark, K. (1998). Performing Architecture Tradeoff Analysis: *ACM Isa w3 Orlando, Florida. CA Software Engineering Institute, Carnegie Mellon University Pittsburgh, PA, U.S.A. 15213.* Retrieved on 5th June, 2010 from www.sei.cmu.edu/reports/04tr018.pdf

Kazman, R., Klein, M., & Clements, P. (2000a). ATAM:Method for Architecture Evaluation. *CA Software Engineering Institute, Carnegie Mellon University Pittsburgh, PA, U.S.A. 15213*. Retrieved on 13th June, 2010 from www.sei.cmu.edu/reports/00tr004.pdf

Kazman, R., Klein, M. & Clements, P., (2002b). "Evaluating Software Architectures: Methods and Case Studies", SEI Series in Software Engineering (2nd ed.): Addison-Wesley Publication.

Kazman, R., Klein, M., Barbacci, M., Longstaff, T., Lipson, H., & Carriere, J. (1998). Experience with Performing Architecture Tradeoff Analysis: *ICSE '99 Los Angeles CA Software Engineering Institute, Carnegie Mellon University Pittsburgh, PA, U.S.A. 15213*. Retrieved on 5th June, 2010 from www. *citeseerx.ist.psu.edu/viewdoc/download?doi=10.1.1.86.2016&rep....*

Kruchten, P., (1995). Architectural Blueprints: The "4+1" View Model of Software Architecture. Retrieved on 2nd June, 2010 from www.cs.ubc.ca/~gregor/teaching/papers/4+1view-architecture.pdf

Kruchten, P., Obbink, H., & Stafford, J. (2006). The Past, Present, and Future of Software Architecture. IEEE Software archive. Volume 23, Issue 2., IEEE 2006.

Lawrence, G. J., & Anthony, J. L. (2001). Using the Architecture Tradeoff Analysis Method to Evaluate a Wargame Simulation System: *A Case Study. Carnegie Mellon University*. Retrieved on 11th June, 2010 from www.sei.**cmu**.edu/reports/01tn022.pdf

Liliana, D., & Eila, N. (2002). A Survey on Software Architecture Analysis Methods. IEEE transactions on software engineering, vol. 28, no. 7, july 2002. IEEE 2006.

Lee, Y., & Choi, H. (2005). Experience of Combining Qualitative and Quantitative analysis Method for Evaluating Software Architecture. *Proceedings of the Fourth Annual ACIS Conference on Computer and Information Sciences (ICIS'05).* IEEE 2005.

Liu, X., El Saddik, A., & Georganas, N.D. (2003). An implementable architecture of an e-learning system. IEEE 2003.

Maier, M.W., Emery, D & Hilliard, R. (2001). Software Architecture: Introducing IEEE Standard 1471. IEEE 2001. Retrieved on 5th June, 2010 from www.ieeexplore.ieee.org/iel5/2/19820/00917550.pdf

Malan, R. & Bredemeyer, D. (2005). "The Role of the Architect", white paper published on the *Resources for Software Architects*. Retrieved on 5th June, 2010 from http://www.bredemeyer.com/papers.htm

Mario, B., Paul, C., Anthony, L., Linda, N., & William, W. (2003). Using the Architecture Tradeoff Analysis Method (ATAM) to Evaluate the Software Architecture for a Product Line of Avionics Systems: A Case Study *Carnegie Mellon University.*

Merson, P., & Smith, D (2003). Using Architecture Evaluation to Prepare a Large Web Based System for Evolution. *Proceedings of the Fifth IEEE International Workshop on Web Site Evolution (WSE'03)*. IEEE 2003.

Nobuo, F., Toru, N., Noriki, A., Hiroki, K., & Kanako, U. (2010). Software Architecture and New Functions in Learning Management System "NOBASU". *Proceeding of the international multi conference of Engineers and computer scientist 2010 Vol 1, IMECS, March 17- 19, 2010, Hong Kong.* IMECS 2010

Nord, R.L., Wood, W.G., & Clements, C.P (2004). Integrating the Quality Attribute Workshop (QAW) and the Attribute-Driven Design (ADD) Method. Retrieved on 10th June, 2010 from www.sei.cmu.edu/library/abstracts/reports/04tn017.cfm

Piyush, M., & Albert, T. (2004). Supporting ATAM with a collaborative Web-based software architecture evaluation tool: *School of Computer Science and Engineering, the University of New South Wales, Sydney NSW 2052, Australia.* Retrieved on 5th June, 2010 from www.linkinghub.elsevier.com/retrieve/pii/S0167642304001947

Punch, K.F. (2006). Developing effective research proposal, (2nd ed.). London: SAGE Publication Ltd.

Shaw, M. (2001). The Coming-of-Age of Software Architecture Research. *Institute for Software Research, International Carnegie Mellon University.* Retrieved on 7th June, 2010 from www. *citeseerx.ist.psu.edu/viewdoc/download?doi=10.1.1.22.*

Som, N. (2006). E-learning: A Guidebook of Principles, Procedures and Practices, 2nd Revised Edition, CEMCA, 2006. Retrieved on 5th June, 2010 from www.cemca.org/e-learning_guidebook.pdf

UUM LearningZone. (2009). what is learning zone? Retrieved on 1st June, 2010 from www.lamanpk.uum.edu.my/index.php?option=com...id